

CS 585 Fall 2026 Programming Assignment #01

Points: 100

Instructions:

1. Place **all your deliverables (as described below) into a single ZIP** file named:

`LastName_FirstName_CS585_Programming01.zip`

2. Submit it to Canvas Assignments section before the due date. **No late submissions will be accepted.**

Objectives:

1. (100 points) Implement the Byte Pair Encoding tokenization algorithm.

Task:

Your task is to implement the Byte Pair Encoding (BPE) tokenization algorithm (both Learner and Segmenter parts).

Deliverables:

Your submission should include:

- Python code file(s). Your main .py file should be named:

`CS585_P01_XXXXXXXXX.py`

where XXXXXXXXXX is your IIT A number (**this is REQUIRED!**). If your solution uses multiple Python files, makes sure that the main (the one that will be run to solve the problem) is named that way and others include your IIT A number in their names as well (for example: `CS585_P01_XXXXXXXXX_EXTRAS.py`).

- This document with your observations and conclusions. You should save it as:

`LastName_FirstName_CS585_P01.pdf`

Implementation:

Your task is to implement (**from scratch – you can't use existing Python package tokenizers | you can ONLY use re and pandas [or similar-subject to approval] packages**) the BPE tokenizer (as explained in class) and apply it to tokenize text.

Your program should:

- Accept three (3) command line arguments, i.e. so your code could be executed with

```
python CS585_P01_XXXXXXXXX.py K TRAIN_FILE TEST_FILE
```

where:

- `CS585_P01_XXXXXXXXX.py` is your python code file name,
- `K` is the number of byte pair merges (a positive integer),
- `TRAIN_FILE` is an input file name corresponding to a training text file that your BPE learner will use to train / build its vocabulary, while `TEST_FILE` is the test text file with some sentences to be tokenized by your TRAINED BPE (its Segmenter component).

Assumptions:

- ◆ both input files are ALWAYS in the same folder/directory as your main python file (i.e. no need to provide/analyze path, just the file name),
- ◆ both input files MAY contain characters outside of the INITIAL vocabulary V (as described below).

Example:

```
python CS585_P01_A11111111.py 20 TRAIN.TXT TEST.TXT
```

Parameter checks:

- If the number of arguments provided is NOT three (none, four or more) display an error message and exit.
- If one of the files does NOT exist in the same folder / directory, display error message and exit,
- If the `K` argument is out of the specified range, assume that the value for `K` is 5.
- Load and process training data file:
 - treat every input file as a single sentence, even if it is made of many (no sentence segmentation needed),
 - remove all punctuation if any,
 - you may need to clean up the input file first to remove all non-printable characters other than space (to be used for space-based tokenization step) and other characters that are NOT in the INITIAL vocabulary V .
- Train your BPE learner on your data set (`TRAIN_FILE`):
 - assume that INITIAL vocabulary V is the set of
 - ◆ all letters (uppercase, lowercase) in English alphabet,
 - add the stop token character to your vocabulary,

- train your BPE Learner by performing K merges,
- this will be your **FINAL** vocabulary V to be used by the BPE Segmenter to tokenize test file text.
- Load and process test data file (`TEST_FILE`):
 - treat every input file as a single sentence, even if it is made of many (no segmentation needed),
 - remove all punctuation if any,
 - you may need to clean up the input file first to remove all non-printable characters other than space (to be used for space-based tokenization step) and other characters that are NOT in the INITIAL vocabulary V .
- Tokenize test data using your trained BPE Segmenter.
- Save results to disk (in the same folder / dictionary as your main .py file):
 - **FINAL** vocabulary V should be saved (one token per line in order of merges; no additional characters) to a text file:

`CS585_P01_XXXXXXXXX_VOCAB.txt`

- Complete result of `TEST_FILE` content tokenization should be saved to a text file:

`CS585_P01_XXXXXXXXX_RESULT.txt`

where: XXXXXXXXX is your IIT A number (**this is REQUIRED!**).

Your program on-screen output should look like this

```
Last Name, First Name, XXXXXXXXX solution:
Number of merges: K
Training file name: TRAIN_FILE
Test file name: TEST_FILE
```

```
Training time: yyy seconds
Tokenization time: zzz seconds
```

```
Tokenization result: <tokenization result here>
```

Where:

- `K`, `TRAIN_FILE`, and `TEST_FILE` are command line inputs listed above,
- `yyy` the actual time (in seconds) it took to generate FINAL vocabulary V (excluding pre-processing),
- `zzz` the actual time (in seconds) it took to tokenize `TEST_FILE` data/text (excluding pre-processing) FINAL vocabulary V ,
- `<tokenization result here>` is actual BPE tokenized test file `TEST_FILE` data/text with space used as a separator.

- If tokenization result is compromised of more than 20 tokens, stop displaying after the 20th token and add the “Tokenized text is longer than 20 tokens” underneath

Analysis, summary, and conclusions:

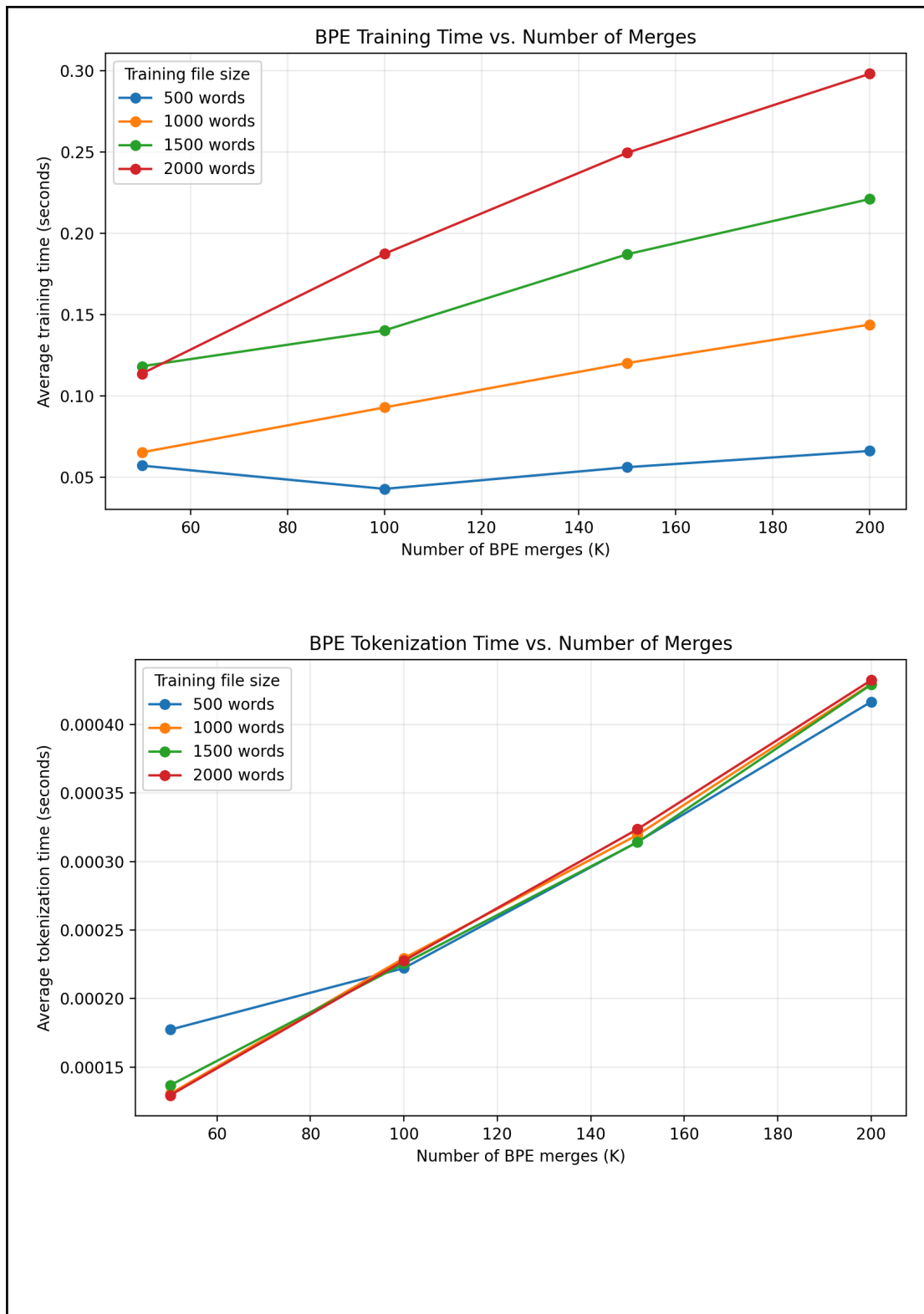
Run the algorithm

- for $K = 50, 100, 150, 200$
- and for every value of K above, change `TRAIN_FILE` size: 500, 1000, 1500, 2000 tokens (words). If sample files not provided, create your own.,
- OPTIONAL: feel free to run the tests on higher values of K and `TRAIN_FILE` size.

When measuring run time for this analysis, you may need to run the same process a number of times and calculate the average. Feel free to create a separate, modified version of your code for that purpose that you DON'T need to submit.

What are your observations and conclusions? How did the number of merges affect the tokenization results (morphemes, prefixes, suffixes, common words discovered or not)? Add comparison plots below.

Summary / observations / conclusions



BPE_analysis_summary

K	Training size	Avg training time (s)	Avg tokenization time (s)
50	500	0.057129566976800600	0.00017730030231177800
50	1000	0.06530738337896770	0.00013045016676187500
50	1500	0.1182465247809890	0.00013669994659721900
50	2000	0.1137283998541530	0.00012939143925905200
100	500	0.04281675014644860	0.00022221682593226400
100	1000	0.09294754979200660	0.0002294668462127450
100	1500	0.14032233338803100	0.00022552497684955600
100	2000	0.18746013357304000	0.00022765863686800000
150	500	0.05617259996943180	0.0003143667709082370
150	1000	0.12025555046275300	0.00031950799748301500
150	1500	0.1871650001965460	0.00031416681595146700
150	2000	0.2496029666159300	0.0003237000200897460
200	500	0.0661570833530277	0.000416700029745698
200	1000	0.14390774997882500	0.00042960839346051200
200	1500	0.2211572247557340	0.00042916662059724300
200	2000	0.29819790823385100	0.00043244208209216600

The results show that training time generally increases when the number of merges and training file size increase. Tokenization time is much smaller, but it also grows when K becomes larger. More BPE merges create larger subword tokens, while fewer merges leave more single characters. Some learned tokens look like common word parts such as suffixes, but BPE learns them from frequency, not from linguistic meaning.

